Tab 1

Title Page

Table of contents

1: Introduction

Background to machine learning and implementation of it in credit scores

There are times where we want to predict outcomes or an output from repetitive tasks based on a set of clues available to us. These clues, which we can call features, are characteristics tied to an individual input data point. Rather than manually having to decode each one, we use machine learning tools to classify data points based on patterns learned from data presented beforehand. This ability to automate and generalize decision-making is at the basis of supervised machine learning, and it has found widespread applications across industries: as explored in this report, credit risk assessment.

In the financial and banking sector, the ability to accurately assess whether a client represents a credit risk is quite important as it allows them to decide whether to allow or deny credit applications. Rather than manually predicting if borrowers present a risk, the rise of machine learning allows us to build models capable of learning from historical data and making these predictions in a consistent and data-driven manner.

In this project I will be applying different classification algorithms to the German Credit Score dataset, which contains information about bank clients and labels each as either a good or bad credit risk. Each client entry is described by a set of features, which will be used as data point reference and clues as to how reliable the client is at paying back. The task is therefore a binary classification problem: **given a client's features, predict whether they represent a good or bad credit risk**.

Objectives

The objectives of this project are:

- To build and train distinct classification models: **Logistic Regression**, **Support Vector Machine**, and **K-Nearest Neighbors** on the German Credit Score dataset.
- To evaluate and compare the predictive performance of each model using: **accuracy**, **precision**, **recall**, and **F1-score**.
- To apply data preprocessing techniques in order to prepare the dataset for each classifier as well as to draw early observations of distributions
- To personally experience each algorithm hands on and draw my own conclusions as to the strengths and limitations of each approach.

Methodology

The project follows a supervised learning pipeline in Python. The German Credit Score dataset is first loaded and explored through an initial analysis phase, which involves examining feature distributions, identifying missing values, and understanding the class balance.

Categorical features are encoded using appropriate techniques, and numerical features are normalized or standardized as required by the models.

The dataset is then split into training and testing subsets to enable unbiased evaluation. Each of the three classification models is trained on the training set and evaluated on the held-out test set. Hyperparameter tuning is carried out where appropriate: selecting the optimal value of k for KNN or tuning the regularization parameter for Logistic Regression and SVM. The models are compared based on their classification metrics, and conclusions are drawn regarding which model best fits the structure and characteristics of this dataset.

2: Downloading Data

The first step to this project is to download the data into python. We import the whole data set into the variable Xy, which we then separate. We locate the features of each client into variable X and the outcome of each client (having a good or bad credit score) into variable y, both being data frames where each index represents the same client.

The following shows the code used for this process:

```
#This imports all the data
Xy = fetch_openml(data_id=31, as_frame=True)

#Here we have the features of each client, the columns being features
and each row representing a new person
X = Xy.data

#gives output of either a good or bad cr+edit score for each customer
y_raw = Xy.target
```

3: Data Overview, Analysis and Processing

3.1: Data Overview

Once imported the data, the first thing we want to see is how many data points and features we have to work with. This would be done by figuring out the shape of the data frame X, where the rows count represents the number of borrowers and the column count represents the number of features in the set.

```
#Data set shape: How many rows (Data points) and columns (Features)
row, col = X.shape

print("There are", col,"features and", row, "data points, specifically
20 characteristics that describe 1000 clients")
```

```
There are 20 features and 1000 data points, specifically 20
characteristics that describe 1000 clients.
```

This shows that we are working with a dataset of **moderate size**, consisting of 1000 applicants described by 20 distinct features. These features will later serve as the **input variables** used by the model to learn patterns and construct a decision boundary. In the case of linear models, this directly contributes to the formation of the **weight vector**, where each feature is assigned a coefficient.

The next step is to inspect the **type of each feature**, as this determines how the data must be processed before being used in a machine learning model. In particular, it is important to distinguish between **categorical** and **numerical** variables. This can be done using the `dtypes` function:

```
#Check which features are numerical and which are categorical, (feature
types)
X.dtypes
```

checking_status	category
duration	int64
credit_history	category
purpose	category
credit_amount	int64
savings_status	category
employment	category
installment_commitment	int64
personal_status	category
other_parties	category
residence_since	int64
property_magnitude	category
age	int64
other_payment_plans	category
housing	category
existing_credits	int64
job	category
num_dependents	int64
own_telephone	category
foreign_worker	category

dtype: object

From the output, we observe that the dataset contains a **mixture of categorical and numerical features**:

- Numerical features (e.g. `duration`, `credit_amount`, `age`)
- Categorical features (e.g. `checking_status`, `credit_history`, `housing`)

This distinction is important because these two types of variables require **different preprocessing techniques**:

- Numerical features will typically be **scaled**
- Categorical features will need to be **encoded into numerical form**

This step is therefore essential for designing a proper preprocessing pipeline later on.

Following this, it is useful to look at the **statistical properties of the numerical features**, such as the mean, standard deviation, and quartiles. This helps identify the **range, spread, and potential outliers** in the data.

```
#Function to give details on numeric values

X.describe()
```

```
  duration  credit_amount  installment_commitment  residence_since
count  1000.000000    1000.000000             1000.000000      1000.00
mean     20.903000    3271.258000                2.973000         2.845000
std      12.058814    2822.736876                1.118715         1.103718
min       4.000000     250.000000                1.000000         1.000000
25%      12.000000    1365.500000                2.000000         2.000000
50%      18.000000    2319.500000                3.000000         3.000000
75%      24.000000    3972.250000                4.000000         4.000000
max      72.000000   18424.000000                4.000000         4.000000



              age  existing_credits  num_dependents
count  1000.000000       1000.000000     1000.000000
mean     35.546000          1.407000        1.155000
std      11.375469          0.577654        0.362086
min      19.000000          1.000000        1.000000
25%      27.000000          1.000000        1.000000
50%      33.000000          1.000000        1.000000
```

```
75%       42.000000          2.000000          1.000000
max       75.000000          4.000000          2.000000
```

From the summary statistics, several observations can be made:

- The feature `credit_amount` has a **large range** (from 250 to 18424), indicating high variability and potential skewness.
- The feature `duration` varies between 4 and 72 months, showing a wide spread in loan lengths.
- The average age of applicants is around 35 years, with values ranging from 19 to 75.
- Some features, such as `installment_commitment` and `num_dependents`, have a relatively **small range**, suggesting limited variability.

These differences in scale between features indicate that **feature scaling will be necessary**, especially when using linear classification models, as large-valued features could otherwise dominate the learning process.

Finally, it is important to verify the **data quality**, in particular whether there are any missing values in the dataset. Missing values can negatively affect model performance and often require imputation.

After checking for null values, it was confirmed that:

- There are no missing values present in the dataset.

This means that no imputation is required, and the dataset is **complete and ready for preprocessing**.

3.2 Exploratory Data Analysis

Before computing the separators, it seemed important to see distinct distributions presented by each of the different classes through plots.

The first being whether the data set is balanced or not:

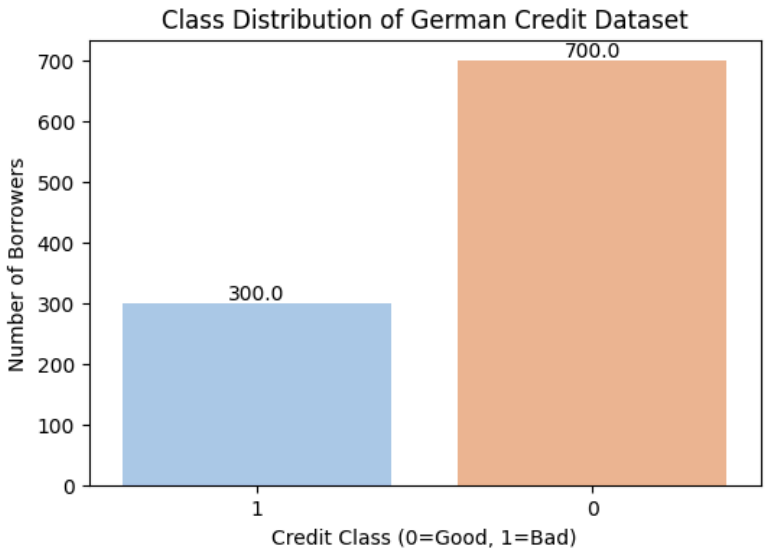


Figure 3.2.1 Count of different credit class

This tells us that there are significantly more users with good credit scores than bad, 700 to 300 to be exact. This is important to note because

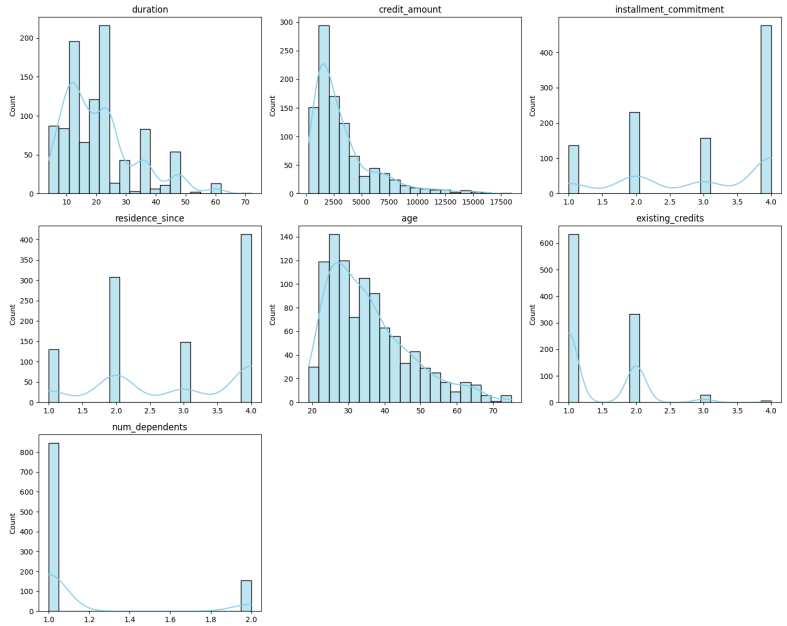


Figure 3.2.2 Hist plot of numeric features

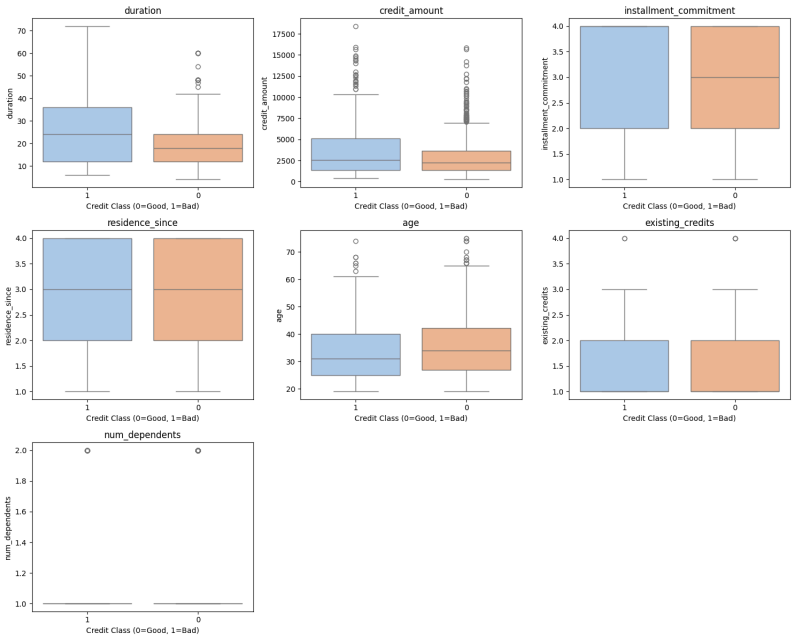


Figure 3.2.3 Boxplots of numeric features by class

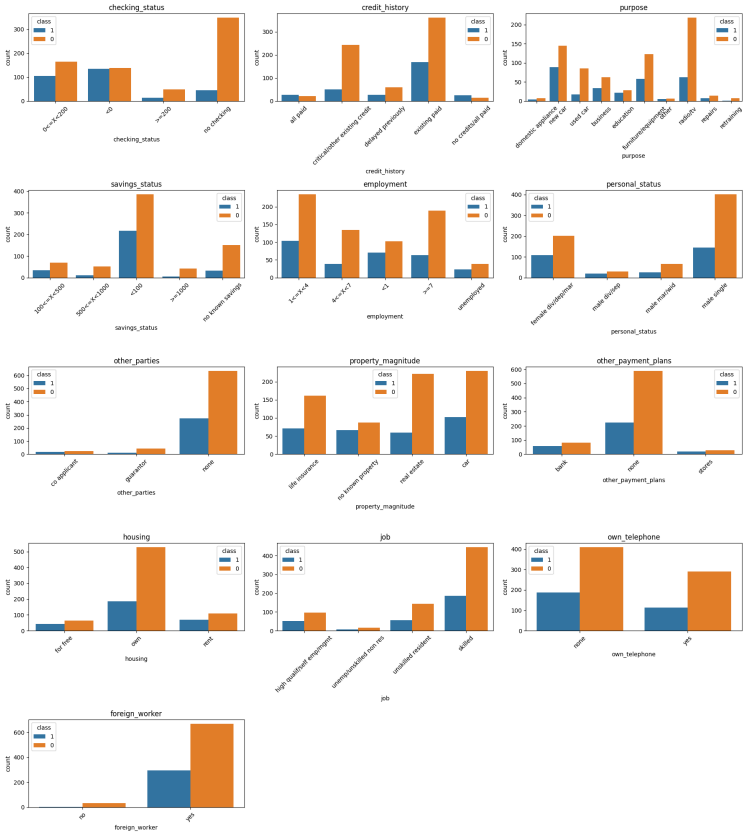


Figure 3.2.4 categorical feature vs target

4: Perceptron explanation

5: Linear model building

5.1: Logistic Model

5.2: Support Vector Machine

6: K-Nearest Neighbors Model Building

7: Comparison Between Models

8: Discussion Questions